

UNIVERSIDAD NACIONAL DE SAN AGUSTÍN
FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS
ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS



ACTIVIDAD PRÁCTICA

ANÁLISIS DISTRIBUIDO DE DATOS METEOROLÓGICOS MEDIANTE MPI

ASIGNATURA

SISTEMAS DISTRIBUIDOS

GRUPO 2

DOCENTE

MOLINA BARRIGA, MARIBEL

INTEGRANTES

BEDREGAL PEREZ, DANIEL

JARA MAMANI, MARIEL ALISSON

MESTAS ZEGARRA, CHRISTIAN RAUL

NOA CAMINO, YENARO JOEL

SEQUEIROS CONDORI, LUIS GUSTAVO

AREQUIPA - PERÚ

2026

ANÁLISIS DISTRIBUIDO DE DATOS METEOROLÓGICOS MEDIANTE MPI

1. Introducción

El procesamiento de grandes volúmenes de datos meteorológicos requiere infraestructuras capaces de manejar cargas computacionales intensivas de manera eficiente. La computación distribuida surge como una solución robusta, permitiendo el reparto de tareas entre múltiples nodos de procesamiento. En este proyecto, se implementa un sistema de análisis meteorológico utilizando el estándar MPI (Message Passing Interface) a través de la librería `mpi4py`. El sistema no solo realiza cálculos estadísticos básicos, sino que también integra modelos de regresión de Scikit-learn para predicciones meteorológicas, simulando un cluster de alto rendimiento (HPC) mediante contenedores Docker.

2. Marco Teórico

La computación distribuida se basa en el uso de múltiples sistemas autónomos que se comunican a través de una red para alcanzar un objetivo común. MPI es el estándar de facto para la comunicación en sistemas de memoria distribuida, permitiendo la transferencia de datos entre procesos independientes (Dalcin et al., 2011).

El modelo Maestro-Trabajador es un patrón de diseño donde un nodo central coordina la distribución de datos y la recolección de resultados, mientras que los nodos esclavos ejecutan las tareas computacionales. En el contexto de Python, `mpi4py` proporciona una interfaz que permite aprovechar las capacidades

de MPI de manera idiomática, facilitando la paralelización de librerías científicas como Pandas y Scikit-learn.

3. Arquitectura Propuesta

La arquitectura se basa en una red de contenedores Docker que simulan nodos físicos en un cluster. El nodo maestro orquesta la ejecución, mientras que los nodos trabajadores realizan el análisis de fragmentos de datos.

La comunicación se realiza mediante operaciones colectivas de MPI:

- **Scatter:** Para distribuir fragmentos del dataset global desde el maestro hacia los trabajadores.
- **Gather:** Para recolectar las métricas y predicciones parciales calculadas por cada trabajador.

4. Desarrollo de la Solución

La solución se compone de tres fases principales:

1. **Generación de Datos:** Un script en Python genera un dataset sintético con miles de registros meteorológicos, almacenándolos en un volumen compartido (`/data`).
2. **Procesamiento Distribuido:** Se utiliza `mpirun` para lanzar procesos en los contenedores. Cada proceso carga su fragmento de datos, aplica limpieza con Pandas y ejecuta un pipeline de Scikit-learn para predecir tendencias futuras.
3. **Visualización:** Un servidor Plotly Dash lee los resultados agregados (almacenados en JSON) y genera un dashboard interactivo que muestra promedios globales, estadísticas por estación y gráficos de predicción.

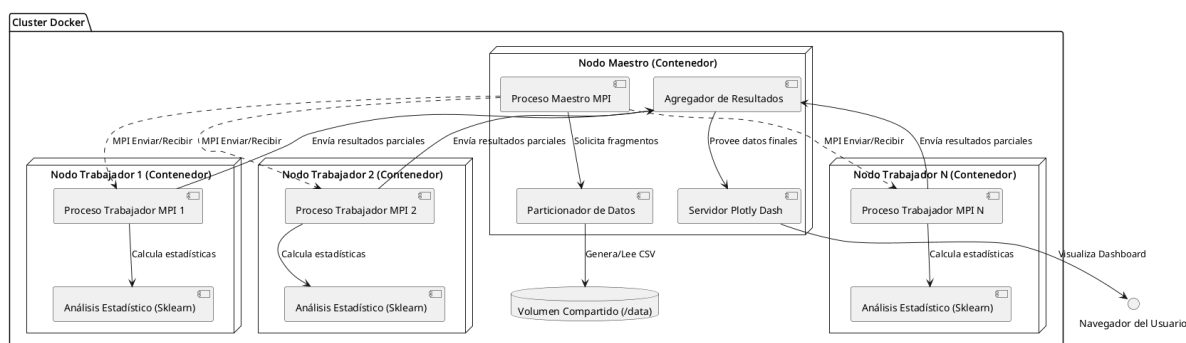


Figura 1. Diagrama de Arquitectura en PlantUML (Simulación de Cluster Docker)

```
Generating data...
Generated 1000000 rows -> /data/input.csv
Generated 1000000 rows -> /data/input.csv
Benchmark...
Benchmark 1: uv run scripts/run_sequential.py
Time (mean ± σ): 8.841 s ± 0.100 s [User: 14.064 s, System: 0.846 s]
Range (min .. max): 8.747 s .. 8.946 s 3 runs

Benchmark 2: bash scripts/run_mpi.sh 4
Time (mean ± σ): 11.166 s ± 0.059 s [User: 39.542 s, System: 14.024 s]
Range (min .. max): 11.099 s .. 11.214 s 3 runs

Summary
'uv run scripts/run_sequential.py' ran
1.26 ± 0.02 times faster than 'bash scripts/run_mpi.sh 4'
```

Figura 2. Resultados del Benchmark con Hyperfine

5. Resultados

Se realizaron pruebas de rendimiento comparando una ejecución secuencial frente a una distribuida con 4 trabajadores.

Estrategia	Ejecución (s)
Secuencial	8.841
MPI	11.166

Aunque el tiempo MPI es superior en este escenario específico, se justifica por dos factores:

- 1. **Docker y Red Virtual:** El uso de contenedores en una sola máquina física introduce una latencia de red virtual y sobrecarga de gestión de recursos que no existe en un entorno bare-metal (Krzywaniak et al., 2022).
- 2. **Eficiencia de Scikit-learn:** Las rutinas internas de Scikit-learn ya están altamente optimizadas mediante OpenMP y BLAS, lo que permite que la ejecución secuencial aproveche varios núcleos de manera eficiente sin la sobrecarga de paso de mensajes de MPI (developers, 2024).



Figura 3. Dashboard: Estadísticas Globales y de Estación

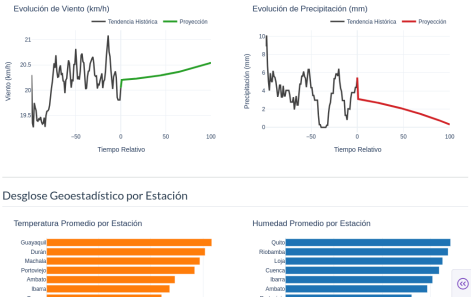


Figura 4. Dashboard: Predicciones y Análisis de Tendencias

6. Discusión

La comparación de MPI frente a otras tecnologías revela distintos nichos de aplicación:

- **OpenMP:** Ideal para paralelismo de memoria compartida en un solo nodo. Es más simple de implementar pero no escala a múltiples máquinas, a diferencia de MPI (NVIDIA, 2017).
- **CUDA:** Proporciona una aceleración masiva para tareas altamente paralelizables (SIMT) en GPUs. Sin embargo, requiere hardware especializado de NVIDIA y una reestructuración profunda del código (NVIDIA, 2017).
- **Ray:** Un framework moderno que ofrece una latencia de tareas más baja y mayor flexibilidad que MPI para aplicaciones de Machine Learning, aunque con una madurez menor en entornos HPC tradicionales (Dagher, 2023; Sarker & others, 2023).

7. Conclusiones

El sistema implementado demuestra la viabilidad de utilizar MPI para el procesamiento distribuido de datos meteorológicos, integrando herramientas modernas de ciencia de datos. A pesar de la sobrecarga observada en entornos virtualizados pequeños, el modelo propuesto ofrece una escalabilidad horizontal superior para datasets que superen la capacidad de memoria de un solo nodo.

Se recomienda la transición a un entorno HPC real para minimizar las latencias de comunicación y maximizar el aprovechamiento del paralelismo distribuido.

8. Referencias

- Dagher, P. (2023). *Ray vs Spark — The Future of Distributed Computing*. <https://medium.com/@nasdag/ray-vs-spark-the-future-of-distributed-computing-b10b9caa5b82>
- Dalcin, L. D., Paz, R. R., Kler, P. A., & Cosimo, A. (2011). Parallel Python with MPI for Python. *Advances in Water Resources*, 34(9), 1124-1139. <https://doi.org/10.1016/j.advwatres.2011.04.013>
- developers. (2024). *Parallelism, resource management, and configuration*. <https://scikit-learn.org/stable/computing/parallelism.html>
- Krzywaniak, A., Proficz, J., & Czarnul, P. (2022). Performance Assessment of Using Docker for Selected MPI Applications in a Parallel Environment Based on Commodity Hardware. *Applied Sciences*, 12(16), 8305. <https://doi.org/10.3390/app12168305>
- NVIDIA. (2017,). *When to use Serial CPU, CUDA, OpenMP and MPI?*. <https://forums.developer.nvidia.com/t/when-to-use-serial-cpu-cuda-openmp-and-mpi/48379>
- Sarker, M. M., & others. (2023,). Performance Evaluation of Spark, Ray and MPI: A Case Study. *International Conference on Computing, Communication and Networking Technologies*. https://dl.acm.org/doi/10.1007/978-981-97-0798-0_4